

COMS W4156 Advanced Software Engineering (ASE)

December 5, 2024

On this day ...

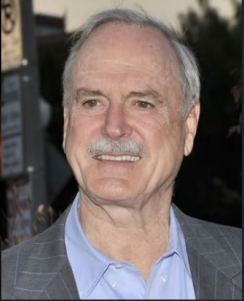


Monty Python's Flying Circus

1974 Final episode of [Monty Python's Flying Circus](#) airs on BBC TV



Comedian and Actor
[Michael Palin](#)



Actor and Comedian
[John Cleese](#)

Agenda

1. Nikita Rajaneesh
2. Sruthi Sudhakar
3. Tao Long
4. if time permits, test generation
5. there's still more to do...



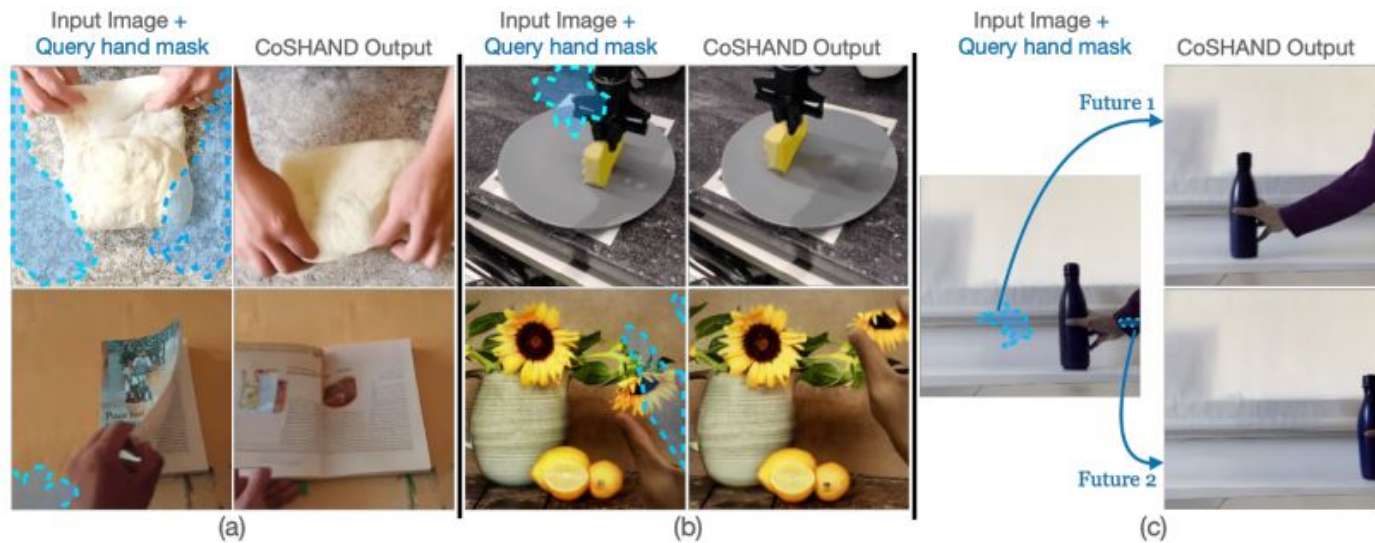
Nikita Rajaneesh

[slides](#)



Sruthi Sudhakar

[slides](#)



Tao Long

slides



Figure 1: Scene 1: Welcome Message



Figure 2: Scene 2: Avatar Customization

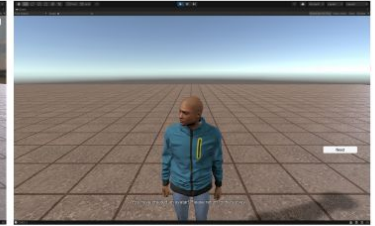


Figure 3: Scene 3: Close-up of Final Avatar



Figure 4: Scene 4: Exclusive Game
(This shows the 3rd person POV version)



Figure 5: Scene 5: Inclusive Game
(This shows the 1st person POV version)

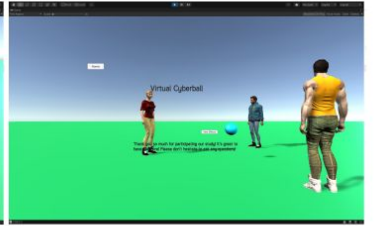


Figure 6: Scene 6: Finish Message

Agenda

1. Nikita Rajaneesh
2. Sruthi Sudhakar
3. Tao Long
4. if time permits, test generation
5. there's still more to do...



Fuzzing (aka Fuzz Testing)

Start with a “seed” input and semi-randomly mutate it (or parts of it), intending to construct malformed or invalid input



Normally applied to relatively long inputs or sequences of inputs like packets, files, and protocols, not to primitive types like integers and strings except as they appear in the fields of larger structured content

Fuzzer leverages mandatory features and constraints of data types and metadata formats, e.g., required fields set to zero length, metadata says the contents are version 3 format but the contents are version 1 (and incompatible with version 3)

Fuzzers usually do not know anything about program semantics, so assertions limited to program crashes, program is very slow, program uses too much memory, etc.

Fuzzing



Fuzz Testing is mostly used to detect security vulnerabilities, so generation tries to produce illegal inputs that make it past any input checkers, validators or sanitizers that typically operate at entry points. *Unchecked* invalid input is how hackers get past defenses and trick software into processing their specially crafted data (“attack vector”)

Feedback/guidance encourages generating new inputs that exercise longer execution paths that reach further into the target software - e.g., after the fuzzer finds a malformed packet header that is accepted by the program, it keeps using that header while fuzzing different parts of the packet body

Easy to implement and use, and yields lots of test inputs - but requires some existing test inputs as seeds

examples: [AFL++](#), [OSS-Fuzz](#)

Free Fuzzing Book

[The Fuzzing Book](#)

From the same lead author as free [The Debugging Book](#)



Automated Test Generation

Metamorphic testing generates both the test input and the assertion (“is it consistent with the predicted output?”) - although it requires some existing inputs to derive from and thus is limited by the quality of the original test suite

Regression, differential and fuzz testing also require an existing test suite

But ideally test generation tools could augment even poor quality or non-existent manually written test cases with high quality test suites



Most Test Generators are Input Generators

The best most test generation techniques can do is generate inputs that are useful for triggering bugs, which is hard, but generating high quality assertions that can accurately detect triggered bugs can be even harder

The assertions typically must be written by human developers OR easy to check generic assertions are used, like did it store the input value, did it crash?

Some test generators assume the current code is correct (🧐), but bugs may be inserted later, so the results of testing the current code are recorded and those results are inserted into the assertions used during regression testing



Random Test Generation

Selects inputs at random from the input space

The test generator need not know anything about equivalence partitioning except for built-in partitions of primitive types, e.g., MinInt, negative, zero, positive, MaxInt

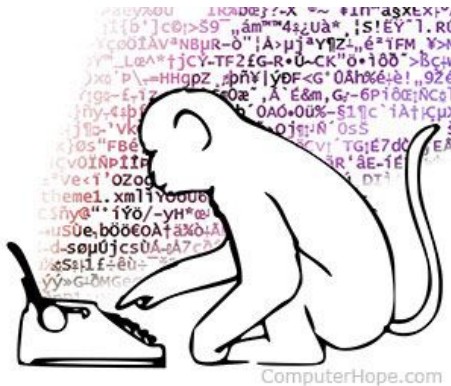
Possible lengths of strings and arrays might be determined by conventional data sizes: empty, one element, four elements, eight elements, 16, 32, 64, 128, 256, etc. elements (stop growing when code under test starts failing)

How is random different from fuzzing? Fuzzing is usually for changing small parts of otherwise valid structured inputs, random is usually for changing entire inputs



Example

```
int myAbs(int x) {  
    if (x > 0) {  
        return x;  
    }  
    else {  
        return x; // bug: should be '-x'  
    }  
}
```



The random tests for this function could be {123, 36, -35, 48, 0}. Of these, only the value '-35' triggers the bug

Most random testing techniques aren't truly random, they use some feedback/guidance mechanism to avoid redundancy and to focus on inputs likely to find bugs

Easy to implement and use, yields lots of test inputs, but not necessarily a "high quality test suite"

example: [Randoop](#)

Agenda

1. Nikita Rajaneesh
2. Sruthi Sudhakar
3. Tao Long
4. if time permits, test generation
5. there's still more to do...



Finishing the Semester

[Second Iteration Demo](#) due December 6 (this Friday)

Second Assessment Friday December 6 12:01AM to Monday December 9 11:59PM (four days starting this Friday)

➤ focus on material October 22 and later

[Demo Day](#) Monday December 16 10AM to 4PM (Monday of finals week)



Ask Me Anything

